SLIDE 1: Title

"Hello everyone, I'm Basil Mohad and today I'll be presenting my machine learning project on Credit Card Fraud Detection for CS4743 Introduction to Machine Learning."

SLIDE 2: Introduction & Problem Statement

"Credit card fraud is a massive problem in digital payments - it causes billions of dollars in losses every year. Actually, I work as a cashier at HEB on weekends and you wouldn't believe how often customers tell me their card was stolen or used fraudulently.

The main challenge with this dataset is extreme class imbalance. Fraudulent transactions make up only about 0.2% of all transactions. This means traditional accuracy metrics are completely misleading.

My goal was to build a model that can accurately identify fraudulent transactions while minimizing false positives. The dataset has over 150,000 transactions with about 30 PCA-transformed features."

SLIDE 3: Data Overview - Class Imbalance

"This bar chart really shows the core challenge. On the left we have about 150,000 legitimate transactions, and on the right - you can barely see it - only around 300 fraudulent ones.

This is severe class imbalance. A naive model that just predicts 'not fraud' for everything would be 99.8% accurate but completely useless because it catches zero fraud.

This is why we can't use accuracy as our metric. We need ROC-AUC instead, which measures how well the model ranks fraudulent transactions higher than legitimate ones."

SLIDE 4: Feature Distributions

"Here we see the distributions of Transaction Amount and Time.

Transaction Amount on the left is heavily right-skewed - most transactions are small, under 500 dollars, but there are outliers up to around 7000.

Transaction Time on the right shows a bimodal pattern representing day and night activity cycles over about 48 hours.

For preprocessing, I used RobustScaler on both features. It uses median and interquartile range instead of mean and standard deviation, making it robust to those outliers we see in the Amount feature."

SLIDE 5: Feature Correlation Analysis

"This correlation matrix heatmap shows relationships between all features. Blue is positive correlation, red is negative.

The key observation is that most features are largely uncorrelated - you see mostly pale colors. This is expected because the features were already PCA-transformed, which creates independent components.

If you look at feat2, it shows strong correlation with our IsFraud target - that dark blue in the corner. This suggests feat2 is an important predictor for detecting fraud."

SLIDE 6: Methodology

"Here's my machine learning pipeline.

Step 1 was Preprocessing with RobustScaler on Time and Amount. Step 2 was an 80/20 stratified train-test split to preserve the class ratio.

Step 3 was my model - ProbaBoost, which is an ensemble of conditional probability classifiers. It uses 30 boosters with 20 buckets each, trained on bootstrapped samples for diversity.

Step 4 was crucial - Threshold Optimization. I'll explain why in the next slides, but the default 0.5 threshold doesn't work for imbalanced data. I searched across the probability range to find the threshold that maximizes F1-score.

Step 5 was evaluation using ROC-AUC, which works well with imbalanced classes and is the competition metric."

SLIDE 7: Results - ROC Curve Performance

"This is the ROC curve showing the model's performance.

The X-axis is False Positive Rate - legitimate transactions incorrectly flagged as fraud. The Y-axis is True Positive Rate - actual frauds we correctly catch. The dashed line is random guessing at 0.5 AUC.

My model achieved an AUC of 0.7361. This means if you randomly pick one fraud and one legitimate transaction, the model correctly ranks the fraud higher 73.6% of the time.

The curve bowing toward the upper-left is what we want - it shows we can catch fraud without too many false alarms. But here's the thing - this good AUC doesn't automatically mean good predictions..."

SLIDE 8: The Problem - Zero Fraud Detected

"So here's what happened with my first approach. I initially tried HistGradientBoosting and Logistic Regression - standard models that usually work well. But look at this confusion matrix - bottom right corner shows zero. The model caught zero fraud cases.

All 54 actual frauds were missed - they're in the bottom-left as false negatives. The model just predicted 'not fraud' for literally everything.

Why? With 99.8% legitimate transactions, the model learned fraud is extremely rare. So it outputs very low probabilities like 0.001 to 0.05 for everything. At the default 0.5 threshold, nothing crosses it.

But here's the key insight - the AUC was still decent! The model CAN rank fraud higher than legitimate transactions. The ranking works, we just need to find the right threshold. So I went back and changed my approach."

SLIDE 9: The Solution - Threshold Optimization

"Here's how I fixed it. Two major changes.

First, I switched to ProbaBoost - an ensemble of conditional probability classifiers trained on bootstrapped samples. This gives more calibrated probability estimates than standard gradient boosting.

Second, and most importantly, I optimized the threshold. Instead of using 0.5, I searched across the entire probability range and picked the threshold that maximizes F1-score.

Look at the results now! We have 11 true positives - we're actually catching fraud! We went from 0 out of 54 to 11 out of 54, that's 20% recall.

Yes, we now have 127 false alarms, but that's an acceptable tradeoff. In real-world fraud detection, investigating 127 false alarms is way cheaper than missing actual fraud that could cost thousands of dollars each."

SLIDE 10: Feature Importance Analysis

"This chart shows permutation importance - how much performance drops when we shuffle each feature.

The top predictors are feat21, feat26, and feat23. These have the biggest impact on detecting fraud. We also see scaled_amount is meaningful, which makes sense since fraud might involve unusual transaction amounts.

Interestingly, feat3 and feat10 at the bottom have negative importance - shuffling them actually improves performance slightly. This suggests they might be adding noise, and removing them could help in future iterations."

SLIDE 11: Future Work & Improvements

"Here are six directions for improvement.

First, Resampling Techniques - SMOTE to generate synthetic fraud samples and balance classes better.

Second, Hyperparameter Tuning - GridSearchCV or Optuna to find optimal settings.

Third, Ensemble Methods - combining XGBoost, LightGBM, and CatBoost through stacking.

Fourth, more Threshold Optimization - maybe optimizing for different metrics depending on business needs.

Fifth, Feature Engineering - creating time-based features like hour of day or weekend flags.

And sixth, Anomaly Detection approaches - treating fraud as outliers using Isolation Forest or Autoencoders might work better given how rare fraud is."

SLIDE 12: Conclusion

"To wrap up: I built a credit card fraud detection model that went through an important iteration.

My first approach with standard models caught zero fraud - a complete failure despite decent AUC scores. This taught me that AUC alone isn't enough for imbalanced problems.

After switching to ProbaBoost and implementing threshold optimization, the model now catches 11 out of 54 frauds - 20% recall - while maintaining 99.6% specificity.

The key takeaway: For imbalanced classification, you must optimize your decision threshold. The default 0.5 almost never works.

Thank you! Any questions?"